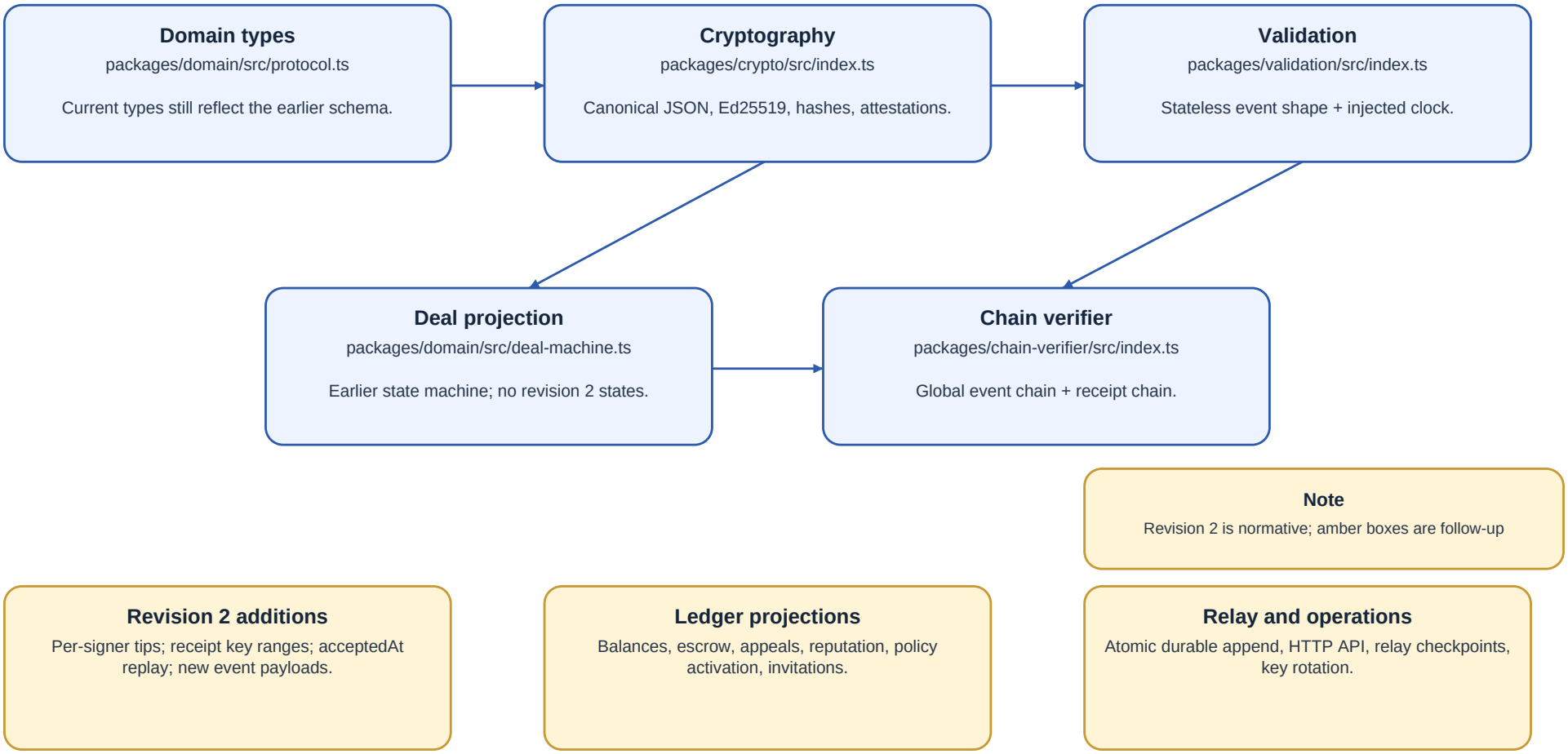
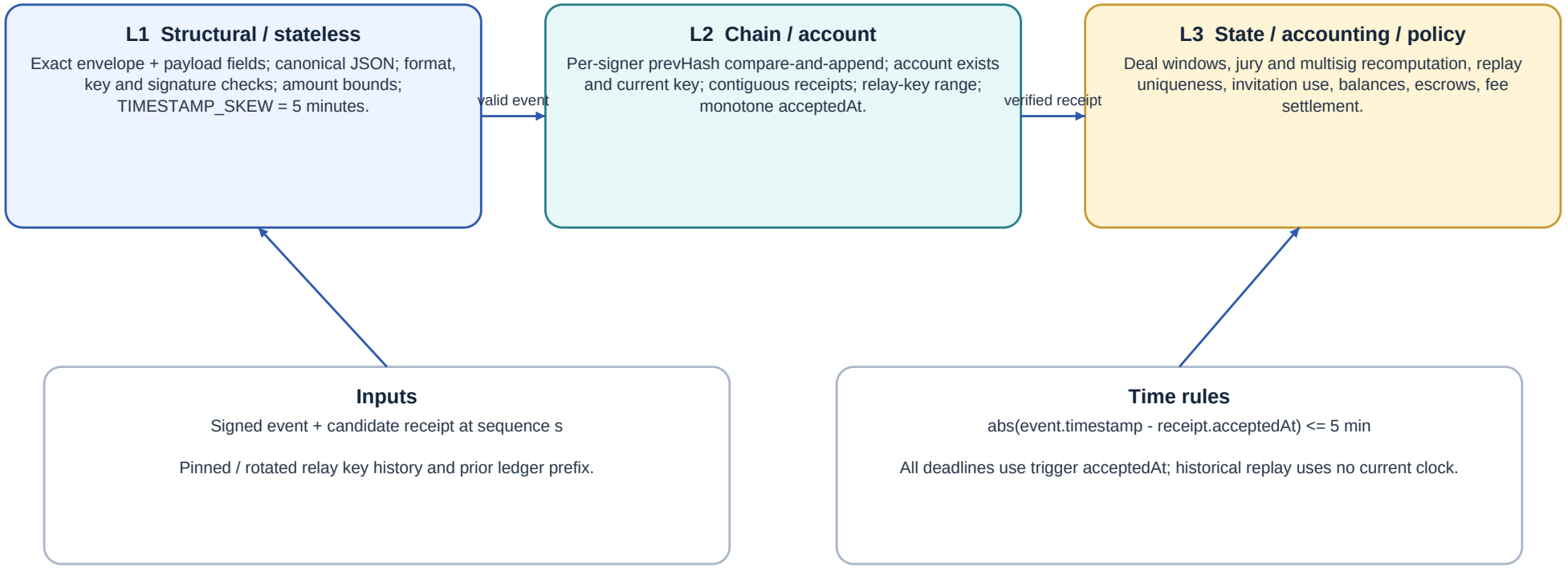
Project map: implemented modules and planned work

Actual repository modules are shown in blue; revision 2 additions not yet implemented are shown in amber.



Validation pipeline: three context layers

Receipt acceptedAt is authoritative; event.timestamp is only a signer claim.

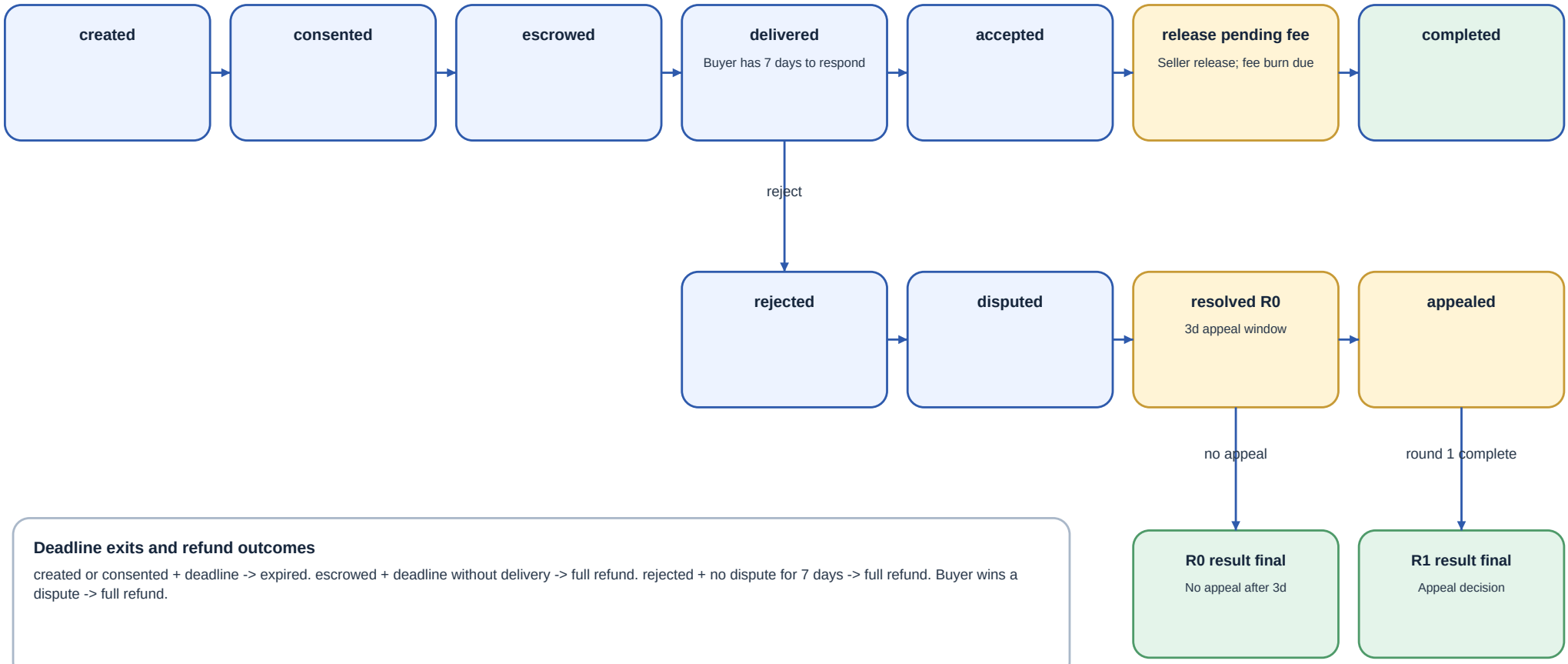


Note

No validation decision reads the machine clock. Policy is selected by the greatest activation_sequence <= the event receipt sequence.

Deal state machine: settlement and dispute paths

All guards are evaluated using receipt.acceptedAt. Terminal states are completed, refunded and expired.



Deadline exits and refund outcomes

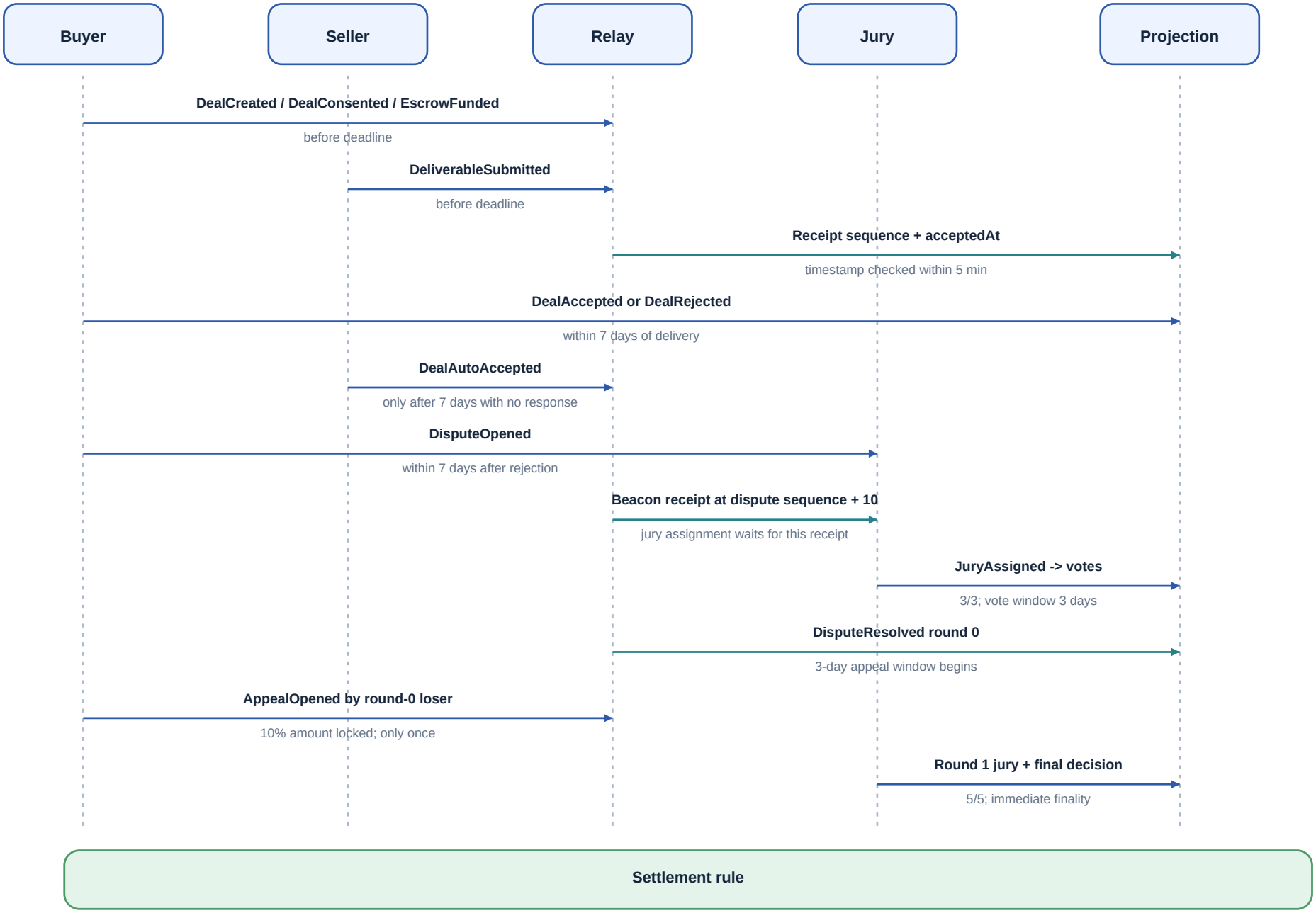
created or consented + deadline -> expired. escrowed + deadline without delivery -> full refund. rejected + no dispute for 7 days -> full refund. Buyer wins a dispute -> full refund.

Settlement by outcome

Seller wins / accepted: release amount = escrow - fee, then FeeBurned = exact fee; same signer; completed only after both. Buyer refund: full escrow, no fee. R0 release waits for appeal window; R1 is final immediately.

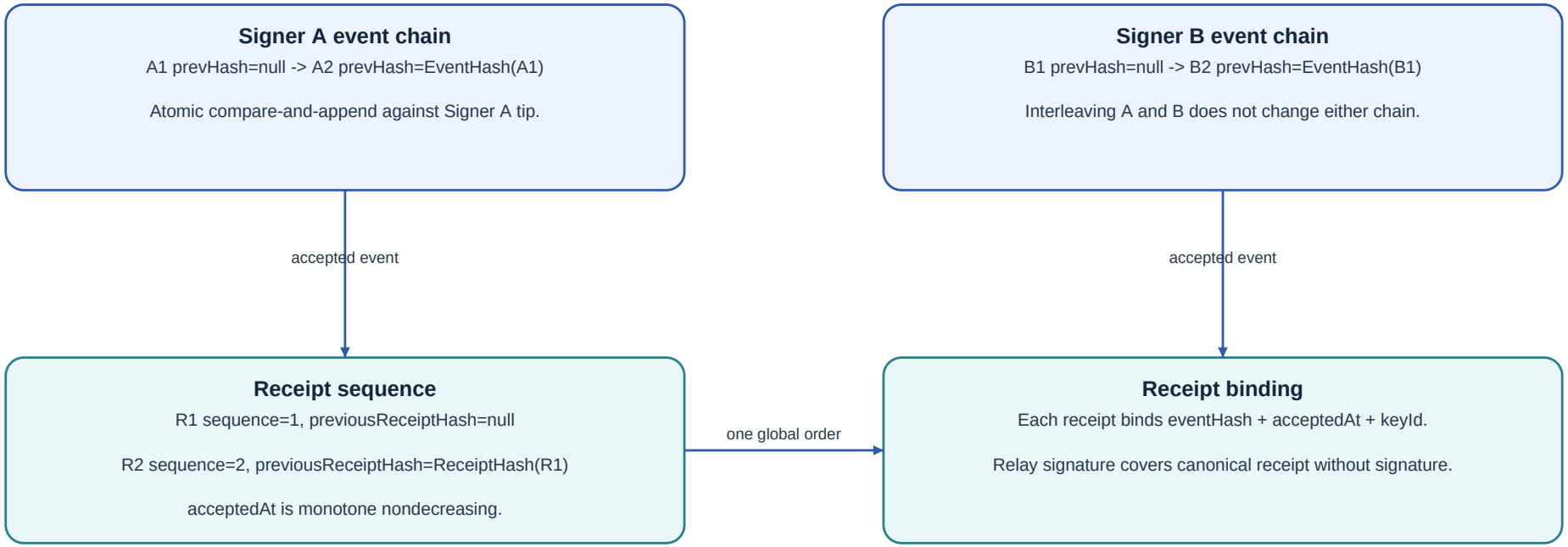
Deal lifecycle: sequence and time gates

A compact sequence view of ordinary acceptance, rejection, dispute and appeal.



Ordering and verification: two linked structures

Per-signer event chains identify each account stream; signed receipts define the single global order.



Verification order

Check event signature and signer tip; receipt sequence and eventHash; previousReceiptHash; relay key valid for sequence; receipt signature; acceptedAt monotonicity.

Note

CHAIN_LINK = stale signer link; FORK = competing append; receipt failures use dedicated receipt codes.

Wire structures: event, receipt and key history

Names and hash/signature relationships follow the revision 2 wire contract.

Note

Global event order comes only from receipt sequence and receipt links. The event prevHash field is never a global link.

Signatures, hashes and commitments

Domain-separated preimages make each signed object and commitment explicit.

Event signature	**Ed25519(UTF-8(canonical(envelope without signature)))**	**Authenticates event fields.**
Event hash	**SHA-256(canonical(complete signed event))**	**Lowercase hex; used by signer prevHash and receipt eventHash.**
Receipt signature	**Ed25519(UTF-8(canonical(receipt without signature)))**	**Relay attests sequence, event binding, acceptedAt and keyId.**
Receipt hash	**SHA-256(canonical(complete signed receipt))**	**Lowercase hex; links receipts and seeds delayed jury beacon.**
Content hash	**SHA-256(raw bytes)**	**Used for deliverables and raw precheck reports.**
Attestation	**Ed25519(UTF-8("skillx/v1/<kind>\n" + canonical(body)))**	**Consent, precheck, vote, treasury, rotation, checkpoint.**
Commitment	**SHA-256(UTF-8(prefix) || 32 salt bytes || content bytes)**	**criteria / reason / purpose; salt may later be revealed.**

Note

Ed25519 verification follows RFC 8032 canonical encoding and small-order rejection; implementations must share generated test vectors.

Who may sign: event authority summary

Signer authority is checked with account state and policy; several events are open to any existing account.

AccountCreated new account pubkey	**JuryAssigned / JurorReplaced** any existing current account; deterministic check
AccountInvited active inviter	**DisputeResolved** any existing current account; tally check
KeyRotated new pubkey; recovery attestation	**EscrowReleased** any existing current account; amount/state check
DealCreated / EscrowFunded buyer	**FeeBurned** same signer as matching seller release
DealConsented / DeliverableSubmitted seller	**AppealOpened** round-0 loser
DealAccepted / DealRejected buyer	**AppealFeeSettled** any existing current account; computed outcome
DealAutoAccepted seller after 7d	**PrecheckCompleted** checker_id; listed checker effective
DealExpired / DisputeOpened either deal party	**TreasurySpend / CurrencyIssued** one of >=2 active-policy approvers
JuryVoted assigned juror	**RelayKeyRotated / PolicyUpdated** one of >=2 active-policy approvers
ReputationEvent / EvidenceRevealed account / deal party	**Checkpoint attestation** relay key valid at sequence

Note

Every signer other than account-creation / rotation exceptions must resolve to an existing current account.

Implementation status: protocol target vs repository today

The diagrams describe the normative revision 2 contract; this page prevents target design from being mistaken for shipped code.

Implemented foundation

TypeScript domain wire types (earlier schema)

Canonical JSON + Ed25519 helpers

Stateless event validation (injected now)

Global ordered-chain verifier

Earlier deal state machine

Revision 2 specified, code follow-up needed

Per-signer chains and atomic append

Receipt keyId and relay-key history

acceptedAt replay + timestamp skew

Accounting, fees, appeal and policy projection

New account, jury, evidence and relay events

Not yet a production relay

Durable atomic store, HTTP API, balance authority,

content retention, recovery wallet, and operational

checkpoint gossip are not evidenced by current code.

Reading the other diagrams

Pages 2-8 show the normative revision 2 design.

Page 1 and this page distinguish repository modules

from target modules and unfinished follow-up.

Note

Use revision 2 diagrams for protocol review and implementation planning; use the implementation column as the factual current-code snapshot.